ToneGuide Project Explanation and Viva Guide

NLP Based Text Tone Transformer College Mini Project

This document explains the ToneGuide project in simple language. It can be used while presenting the project or answering short viva questions. The project takes a sentence, studies its language, rewrites it in a selected tone, and compares the original and rewritten text.

Project Summary

ToneGuide is a web application that changes the tone of written text. For example, it can rewrite a casual sentence in a professional, formal, friendly, academic, humorous, emotional, or neutral style. It also shows sentiment, emotion, tone features, and similarity between the two versions.

Item	Explanation
Project name	ToneGuide
Project type	NLP based web application
Main purpose	To rewrite text in a selected tone while trying to keep its meaning.
Input	A sentence or short paragraph, selected tone, and intensity level.
Output	Rewritten text and analysis of both the original and transformed text.

Problem Statement

People often know what they want to say, but they may not know the best tone for a situation. A message to a friend, teacher, manager, or client may need a different style. ToneGuide helps the user change the writing style without manually rewriting the whole message.

Objectives

- Accept text from the user through a simple web page.
- Analyze sentiment, emotion, and language features of the text.
- Rewrite the text in a tone chosen by the user.
- Compare original and transformed text to check meaning similarity.
- Show the results in an understandable form.

How the Project Works

The following flow is used for every request:

Step	What happens
1 User input	The user enters text, chooses a target tone, and sets a tone intensity from 0 to 100.
2 Validation	The app checks that the text is not empty or too long.

Step	What happens
3 Preprocessing	The text is cleaned and basic values such as word count and sentence count are calculated.
4 Original analysis	The app finds sentiment, emotion, and tone related language features.
5 Transformation	The app rewrites the text using an optional API, FLAN T5, or a rule based fallback.
6 New analysis	The transformed text is analyzed again using the same steps.
7 Similarity	The app compares both texts to estimate how much meaning is preserved.
8 Results	The browser displays the rewritten text, analysis, similarity score, and processing time.

Technologies Used

Technology	Use in this project	Simple explanation
Python	Backend language	Python runs the NLP logic and server code.
FastAPI	Web framework	It creates API endpoints so the webpage can send text to Python and receive results.
Uvicorn	ASGI server	It starts and runs the FastAPI application locally.
HTML CSS JavaScript	Frontend	These create the page, styling, buttons, and result display.
NLTK	Text preprocessing	It helps with tokenization and language related processing.
Transformers	Pretrained NLP models	It loads models for sentiment, emotion, and text generation.
Sentence Transformers	Similarity	It converts sentences to number vectors for meaning comparison.
Scikit learn	Fallback similarity	It provides TF IDF cosine similarity if the main similarity model is unavailable.

Important Concepts in Simple Words

Term	Short answer for presentation or viva
NLP	Natural Language Processing is a field of AI that helps computers understand and work with human language.

Term	Short answer for presentation or viva
FastAPI	FastAPI is a Python framework used to build fast web APIs. In this project it receives text from the webpage and sends the NLP result back.
API	An API is a way for two programs to communicate. Here, the HTML page communicates with the Python backend through API endpoints.
Endpoint	An endpoint is a URL that performs a specific task. For example, /api/transform changes the tone of text.
Preprocessing	Preprocessing means cleaning and preparing text before analysis, such as removing extra spaces and counting words.
Tokenization	Tokenization means splitting text into smaller parts such as words or sentences.
Sentiment	Sentiment shows whether the text is positive, negative, or neutral.
Emotion	Emotion identifies feelings such as joy, sadness, anger, fear, surprise, disgust, or neutral.
Tone	Tone is the style or attitude of writing, such as formal, casual, friendly, or professional.
Transformer model	A transformer is a deep learning model that understands the relationship between words in text.
Cosine similarity	It is a score used to compare two text vectors. A higher score means the sentences are closer in meaning.

Project Files and Their Roles

File or folder	Role
app.py	Main FastAPI application. It defines the API endpoints and connects all NLP steps.
index.html	Frontend page where the user enters text and sees results.
nlp preprocessing.py	Cleans text and calculates basic text statistics.
nlp sentiment.py	Finds positive, negative, or neutral sentiment.
nlp emotion.py	Finds the most likely emotion in the text.
nlp tone.py	Uses language features to score the tone of text.
nlp transformer.py	Rewrites text into the selected target tone.
nlp similarity.py	Measures semantic similarity between original and transformed text.
models model_loader.py	Loads and caches the pretrained models so they are not loaded repeatedly.

File or folder	Role
utils helpers.py	Contains input validation and small helper functions.
requirements.txt	Lists the Python libraries needed to run the project.

Models Used

Task	Model or method	What it does
Sentiment	DistilBERT SST 2	Classifies text as positive or negative. The project can map uncertain results to neutral.
Emotion	DistilRoBERTa emotion model	Gives scores for anger, disgust, fear, joy, neutral, sadness, and surprise.
Text transformation	FLAN T5 small	Generates a rewritten sentence from an instruction about the required tone.
Similarity	MiniLM sentence transformer	Creates sentence embeddings and compares them using cosine similarity.
Tone detection	Rule based scoring	Looks at words, contractions, slang, punctuation, emojis, and sentence style.

Tone Transformation Logic

The project follows a safe order for transformation. It first checks whether an optional external text generation API is configured. If not, it tries the local FLAN T5 model. If that model cannot load or gives an empty result, the project uses simple rule based word replacements. This fallback keeps the app useful even when a heavy model is unavailable.

Target tone	Example change
Professional	Hey, send me the files becomes Please send me the files at your convenience.
Formal	I cannot come tomorrow becomes I regret to inform you that I will be unable to attend tomorrow.
Casual	I would like to request the notes becomes Can you send me the notes?
Friendly	Please complete this task becomes Hi! Please complete this task. Thanks so much!
Neutral	This is amazing! becomes This is useful.

The exact wording can change because model based generation is probabilistic. The important aim is to preserve the main message while changing the style.

API Endpoints

Endpoint	Method	Purpose
/	GET	Returns the main index.html page when the app runs locally.
/api/tones	GET	Returns the list of available target tones.
/api/transform	POST	Receives input text, selected tone, and intensity. It returns the transformed text and analysis.

Example Input and Output

Field	Example
Original text	I cannot come to the meeting tomorrow because I have some work.
Target tone	Professional
Intensity	70 percent
Possible output	I will be unable to attend tomorrow's meeting due to a prior commitment.
Similarity	A high score is expected because both sentences have the same main meaning.

Viva Questions and Short Answers

Question	Answer
What is the main aim of this project?	The aim is to rewrite user text in a selected tone and show NLP analysis before and after rewriting.
Why did you use FastAPI?	FastAPI makes it easy to create Python API endpoints. The frontend sends text to these endpoints and receives JSON results.
What is the frontend in this project?	The frontend is index.html. It uses HTML for structure, CSS for design, and JavaScript to call the API and display results.
What is the backend?	The backend is app.py and the Python NLP modules. It performs analysis and transformation.
What is JSON?	JSON is a simple text format used to send structured data between the frontend and backend.
What is sentiment analysis?	It checks whether a sentence expresses a positive, negative, or neutral opinion.
How is emotion detected?	A pretrained emotion classification model gives scores for seven emotion labels, and the highest score is selected.

Question	Answer
How is tone detected?	The project checks explainable language features such as contractions, slang, formal words, punctuation, and emojis.
How is tone changed?	The project gives an instruction to FLAN T5. If it is not available, it uses rule based replacements as a fallback.
Why do you use intensity?	Intensity controls how strongly the selected tone should be applied. A higher value gives a stronger style change.
What is semantic similarity?	It checks whether original and transformed sentences still have similar meaning, even if the words are different.
Why is model caching useful?	Loading a model takes time. Caching keeps a loaded model in memory so later requests are faster.
What happens if a model fails?	The project handles errors and uses a simpler fallback where possible, so the app does not stop completely.
What are the limitations?	Model output may not always be perfect, the first model load can be slow, and larger models need more memory.

How to Demonstrate the Project

- Run the project using python app.py and open http://localhost:8000 in a browser.
- Enter a casual sentence such as Hey bro, send me the slides ASAP.
- Select Formal or Professional and set intensity to 70.
- Click the transform button and explain the rewritten text.
- Show the original and transformed sentiment, emotion, and tone analysis.
- Point out the semantic similarity score to explain that the project tries to preserve meaning.
- Try another tone, such as Friendly or Casual, to show that the output style changes.

Limitations and Future Improvements

- The quality of rewriting depends on the pretrained model and the input sentence.
- The first request can be slower because models may need to download and load into memory.
- The rule based fallback is simple and cannot handle every writing situation.
- A future version can add more languages, user login, saved history, and better fine tuned tone models.
- A future version can let users compare multiple tone outputs at the same time.

Conclusion

ToneGuide shows how NLP can be used in a practical writing tool. It combines text preprocessing, sentiment analysis, emotion detection, rule based tone analysis, text generation, and similarity checking in one simple web application. The project is suitable as a mini project because every step is visible, explainable, and connected to a clear real life use case.